

Phase 3, Test Design

10 prioritised scenarios for the critical journeys Samuel, QA Automation Senior · 15 August 2026 · v1.0

The brief asks for relevance, not exhaustiveness. Each scenario below is here because it closes a specific risk from the Phase 1 matrix. Where I could have written five variations of the same idea, I wrote one and moved to a different failure mode.

Minimum required: 5 high-priority scenarios across ≥ 3 critical journeys. **Delivered:** 6 P1 scenarios across 4 critical journeys (J1 Transfer, J2 Authentication, J3 Ledger, J4 Administration), 39 concrete test cases, of which **21 are negative or boundary**.

Coverage at a glance

Journey	Scenarios	P1	Negative/boundary cases
J1 — Execute a transfer	TS-03, TS-04, TS-05	3	16
J2 — Authenticate & hold session	TS-01, TS-05d	1	3
J3 — Read ledger & KPIs	TS-02, TS-07, TS-08, TS-09	2	2
J4 — Administer users	TS-01, TS-10	1	4
<i>Cross-cutting</i>	TS-06, TS-10c/d	—	—

TS-01, Authorisation boundary on directory endpoints

Journey	J4 Administration, J2 Authentication
Risk closed	R-S1 (25), R-S3 (20)
Rule	R10 — only ADMIN may read the user directory
Technique	Decision table — caller identity × endpoint × expected verdict
Level / type	API · security
Priority	P1
Automated	tests/api/authorization.spec.ts

Decision table

#	Caller	Endpoint	Expected	Observed
1	(none)	GET /users	401	200 + 3 records ☐
2	(none)	GET /contacts	401	200 + 3 records ☐
3	MEMBER	GET /users	403	<i>unverified</i>
4	MANAGER	GET /users	403	<i>unverified</i>
5	ADMIN	GET /users	200	<i>unverified</i>

Data required: none for rows 1-2, *that is the finding*. Rows 3-5 need one valid session per role.

Why the anonymous row comes first: a decision table built only from the three documented actors would never have contained row 1, and row 1 is the defect. The fourth “actor”, the caller with no identity at all, is the one the documentation forgets and the one attackers use.

TS-02, Tenant scoping on the ledger

Journey	J3 Ledger
Risk closed	R-S2 (25)
Rule	Implied by B.3 (“vue individuelle”) — never stated as a scoping rule (C-14)
Technique	Equivalence partitioning on scope parameter + negative testing
Level / type	API · security
Priority	P1
Automated	tests/api/authorization.spec.ts

Partition	Input	Expected	Observed
No scope	GET /transactions	Caller’s records only	All 15, unauthenticated ☐
Own scope	?userId=<self>	Same set	<i>unverified</i>
Foreign scope	?userId=u2	403 or caller’s set	Byte-identical to unscoped ☐
Invalid scope	?userId=nonexistent	400 or empty	<i>unverified</i>

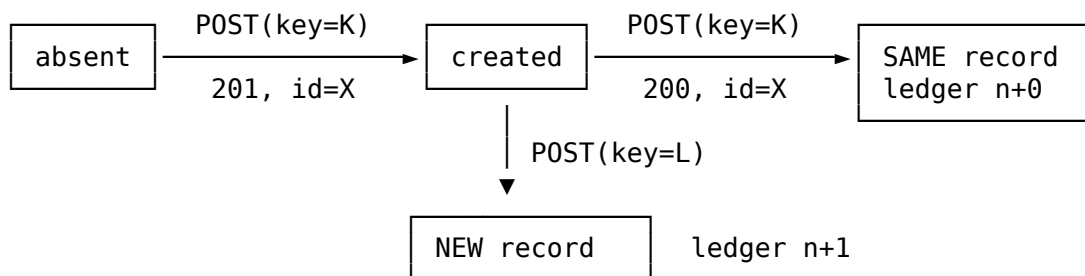
Data required: two accounts with disjoint transactions, **which the seed does not provide**. Every record is global, so I cannot even construct the positive case. That is itself reportable (R-B4).

Design note: the assertion is that the foreign-scope response *differs* from the unscoped one, not that it is empty. That phrasing survives whichever scoping model the team picks, and it fails loudly today because the two are identical.

TS-03, Idempotency of POST /transfers

Journey	J1 Transfer
Risk closed	R-B2 (20)
Rule	None documented — header appears in Annexe F examples only (C-08)
Technique	State transition — the same logical operation submitted twice
Level / type	API · functional / integrity
Priority	P1
Automated	tests/api/transfers-idempotency.spec.ts

State model



Case	Input	Expected
TS-03	Same key, identical body, ×2	One record; replay returns the original id
TS-03b	Two distinct keys	Two records — guards against over-eager dedup
TS-03c	No key at all	400/422 demanding the header (<i>assumption H-04</i>)

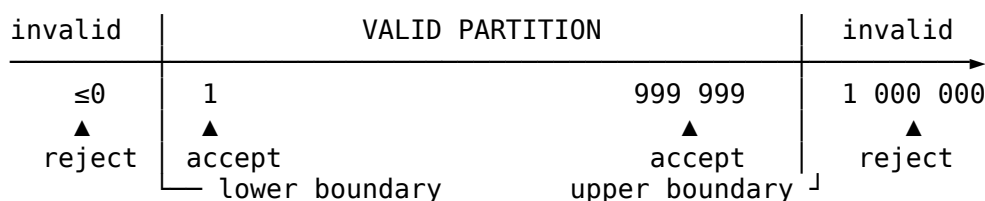
Data required: a valid recipient, an amount well inside the ceiling, a valid PIN, and two UUID keys generated per run so repeat runs never collide.

Why TS-03b exists: without it, a backend that rejected *every* second request regardless of key would pass TS-03. Deduplicating too aggressively is as much a defect as not deduplicating, it silently drops legitimate transfers.

TS-04, Server-side validation of transfer rules

Journey	J1 Transfer
Risk closed	R-B1 (20)
Rules	R01 (future date), R02 (6-digit PIN), R03 (ceiling), R04 (note ≤ 60)
Technique	Boundary value analysis + equivalence partitioning
Level / type	API · functional negative
Priority	P1
Automated	tests/api/transfers-validation.spec.ts

Boundary analysis on amount (cents, per Annexe E, assumption H-02)



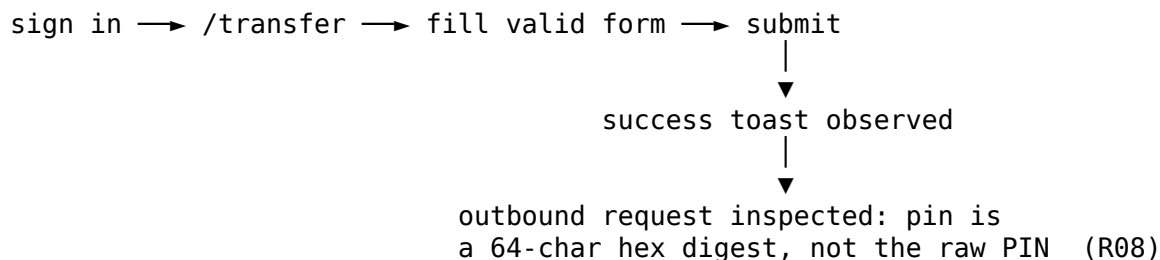
#	Case	Value	Expected
1	On the ceiling	999999 (\$9,999.99)	201
2	One cent over	1000000 (\$10,000.00)	400
3	Far over	100000000 (\$1M)	400 — <i>Annexe H TC-03 got 201</i> □
4	Zero	0	400
5	Negative	-2500	400
6	Sub-unit	12.345	400
7	Note boundary	61 characters	400 (R04)
8	PIN boundary	12345 — 5 digits	400 (R02)
9	Past schedule	yesterday	400 (R01)
10	Missing field	no pin	400

Data required: a valid recipient id; all other values are derived from the rules, not from the seed, so the set stays valid when the demo data changes.

The principle behind all of this: Annexe H shows TC-06 (form rejects a long note) passing while TC-03 (API accepts \$1M) fails. Both target the same product. **UI validation is a usability feature; only the API boundary is a control.** Every Annexe C rule is therefore re-asserted here, regardless of what the form does.

TS-05, Critical journey, end to end

Journey	J1 + J2
Risk closed	R-B1, plus integration risk no single layer covers
Rules	R02, R03, R07, R08
Technique	Use-case / scenario testing on the happy path, boundary on the variants
Level / type	E2E · functional
Priority	P1
Automated	tests/e2e/critical-journey.spec.ts



Case	Priority	Assertion
TS-05	P1	Journey completes; PIN leaves the browser hashed
TS-05b	P2	\$10,000.00 → form shows the R03 error
TS-05c	P2	5-digit PIN → form shows the R02 error
TS-05d	P2	Wrong password → generic error, stays on /login

Data required: MEMBER credentials, an owned contact (c1 Sarah Chen), a small amount (\$12.34, deliberately inside every limit so the test fails only for the reason it is testing), a 6-digit PIN.

Why only one full E2E: everything a second E2E would assert is already asserted at the API level, in milliseconds, with a precise failure message. This one exists to answer the question the API cannot: does the whole chain, routing, guard, form, HTTP layer, backend, ledger, hold for a real user?

Note on R08: TS-05 asserts the hash *is transmitted*, which is what the rule says. It does not assert the control is *effective*, it is not. An unsalted SHA-256 of a 6-digit PIN is enumerable in under a second, and the hash becomes the replayable credential (C-09). A green test and an ineffective control, at the same time. That is why C-09 is in the contradictions list and R-S4 is in the risk matrix, rather than being quietly satisfied by a passing test.

TS-06, Schema and contract validation

Journey	Cross-cutting
Risk closed	R-T1 (15)
Technique	Contract testing — dual-schema (published vs observed)

Level / type	API · contract
Priority	P2
Automated	tests/api/contract.spec.ts

Case	Schema	Expected	Observed
TS-06	/transactions vs observed	Conforms	☐ pass
TS-06b	/transactions.amount integer ≥ 1 (cents) vs published		Signed floats ☐
TS-06c	/users vs observed	Conforms	☐ pass
TS-06d	/contacts.accountMask vs $\wedge^*\{4\}\backslash d\{4\}\$$	Masked	☐ pass
TS-06e	POST /transfers vs published	Conforms to Annexe E	<i>unverified</i>

Data required: none, the endpoints supply their own.

Why two schema families: validating only against the published contract would flag the whole product as broken. Validating only against observed shapes would let a contract breach pass unnoticed forever. Running both makes the *disagreement between them* the finding, which is exactly what DEF-004 is.

TS-06d is a tripwire, not a check. It exists so that the day someone returns a full account number in accountMask, a test goes red before the data reaches a log, Annexe D's "PAN jamais loggé".

TS-07, Cross-endpoint data consistency

Journey	J3 Ledger
Risk closed	R-B3 (16), R-T3
Technique	Invariant / property-based — properties true for any dataset
Level / type	API · data integrity
Priority	P2
Automated	tests/api/data-consistency.spec.ts

Case	Invariant	Result
TS-07	Every non-null recipientId resolves in /contacts	☐ pass
TS-07b	Withdrawals negative, refunds positive (<i>H-06</i>)	☐ pass
TS-07c	The three status filters partition the ledger exactly — no loss, no duplication, totals identical in integer cents	☐ pass
TS-07d	No duplicate transaction ids	☐ pass
TS-07e	Single-user p95 on /transactions < 300 ms	☐ pass

Data required: none, invariants hold for any dataset, which is precisely why they were chosen. The seeded data already contradicts Annexe F.6; a value-pinned test would have been dead on arrival.

TS-07c is the most useful test in this group. A record that belongs to no status filter is invisible in the UI but still counts towards a balance, a silent reconciliation gap, and one of the few ways money genuinely disappears from a dashboard.

TS-07d is the double-debit tripwire. Annexe G extract B produced two settled transfers from one user action. If that reaches the ledger, this test catches it.

TS-08, API ↔ UI reconciliation

Journey	J3 Ledger → J1 Transfer
Risk closed	R-B3 (16)
Technique	Comparison / oracle testing — the API is the oracle
Level / type	E2E · data integrity
Priority	P1
Automated	tests/consistency/api-vs-ui.spec.ts

Case	Assertion
TS-08	Every amount rendered across all pages equals a ledger amount, compared as a multiset in integer cents
TS-08b	The displayed balance equals one of two defensible derivations (<i>blocked on Q2</i>)

Data required: ADMIN session (widest visibility) + the ledger read in the same test.

Two details that decide whether this test is real:

1. **It paginates.** The table shows 5 rows; the ledger has 15. A single-page read would compare 5 against 15 and pass for the wrong reason.
2. **It compares in integer cents, order-independent.** The table may sort differently from the API, not a defect. A missing or altered value is.

Reference point: Annexe F.6 documents a table total of \$44,800 against a Balance KPI of \$48,291, a **\$3,491 unexplained gap**. On the live environment neither figure is reproducible: I compute - **\$389.66** across all 15 records and **\$1,883.14** across completed ones. TS-08b cannot assert until Q2 is answered, and it says so in its own failure message rather than guessing.

TS-09, Filter enum validation

Journey	J3 Ledger
Risk closed	R-T4 (12)
Technique	Equivalence partitioning on the enum domain
Level / type	API · negative
Priority	P2
Automated	tests/api/transfers-validation.spec.ts

Partition	Input	Expected	Observed
Valid	?status=pending	200, only pending	☐ pass
Out of enum	?status=not-a-real-status	400	200 [] ☐
Empty	?status=	400 or ignored	<i>unverified</i>
Injection-shaped	?status=' OR 1=1--	400	<i>unverified</i>

Data required: at least one record per status, satisfied by the seed.

Why 200 [] is worse than 400: a caller cannot distinguish “no matching records” from “your query was malformed”. A typo in a client silently renders an **empty ledger** instead of raising an error. On a financial dashboard, that is the class of bug that hides missing money rather than reporting it.

TS-10, RBAC in the UI, and observability

Journey	J4 Administration + cross-cutting
Risk closed	R-S3 (20), R-O1 (15)
Rule	R10
Technique	Decision table (role × visibility) + existence checks
Level / type	E2E + API · security / operability
Priority	P2 / P3
Automated	tests/e2e/rbac.spec.ts, tests/api/observability.spec.ts

Case	Assertion	Result
TS-10	Admin link visible to ADMIN, hidden from MANAGER and MEMBER	<i>unverified</i>
TS-10b	MEMBER typing /admin directly is refused	<i>unverified</i>
TS-10c	/health, /ready, /metrics return 200	404 ×3 ☐
TS-10d	A correlation id is echoed to the caller	absent ☐

Data required: one session per role.

TS-10 vs TS-10b is the point of this scenario. Hiding a nav link is a usability measure. Refusing the route, and refusing the endpoint behind it, is the control. A product can pass TS-10 and still be wide open, which is precisely what the evidence shows: Annexe H TC-11 has a MEMBER receiving 8 user records from /admin/users.

On TS-10c’s priority: marked P3 because it blocks no functional journey. But Annexe D commits to a 15-minute RTO and 99.9% availability, and **neither is measurable without a liveness probe**, an operator cannot page on a screenshot. It is a P3 that carries an operational reserve into the Go/No-Go.

Traceability

Rule	Covered by	Level
R01 future scheduled date	TS-04 case 9	API
R02 six-digit PIN	TS-04 case 8, TS-05c	API + UI
R03 \$9,999.99 ceiling	TS-04 cases 1-3, TS-05b	API + UI
R04 note ≤ 60 chars	TS-04 case 7	API
R05 completed = read-only	<i>not automated — see note</i>	—
R06 member's own contacts	TS-02, TS-07	API
R07 401 → session invalidated	TS-05d	E2E
R08 PIN hashed client-side	TS-05	E2E
R09 delete needs confirmation	<i>deliberately not automated</i>	—
R10 admin-only endpoints	TS-01, TS-10, TS-10b	API + E2E

Two deliberate omissions:

- **R05** is covered by the incumbent suite (Annexe H TC-07: passing). Duplicating a green test buys nothing; I spent the time on TS-02 instead.
- **R09** is not automated because deleting a financial record on a shared, unresettable environment is irreversible, and because deletion contradicts Annexe D's 7-year immutable retention (C-01). I escalated the contradiction rather than automate a behaviour that may need to be removed entirely.

Implementation in tests/. Execution evidence in docs/05-results-analysis.md.